Predicting Incident Resolution Time

A Data Science Analysis

Yufei Wang

April 27, 2026

Carnegie Mellon University

Table of Contents

Introduction

Project Motivation

- Problem:
  - Predict whether an incident will be resolved **Fast** or **Slow**
  - Based on incident attributes and lifecycle (process) information
- Why it matters:
  - Slow resolution $\rightarrow$ downtime, lower user satisfaction
  - Impacts service quality and operational efficiency
- Who benefits:
  - IT managers (resource allocation)
  - Support teams (prioritization)
- Approach:
  - Machine learning models (LR, DT, RF)
  - Analyze feature selection and process feature impact

Prediction Task

- Task formulation:
  - Binary classification: **Fast** vs **Slow**
  - Target derived from incident resolution time
- Input features:
  - Static attributes (e.g., priority, urgency)
  - Process features (e.g., reassignment, events)
- Evaluation:
  - Accuracy, Macro F1, Weighted F1
  - ROC-AUC, PR-AUC

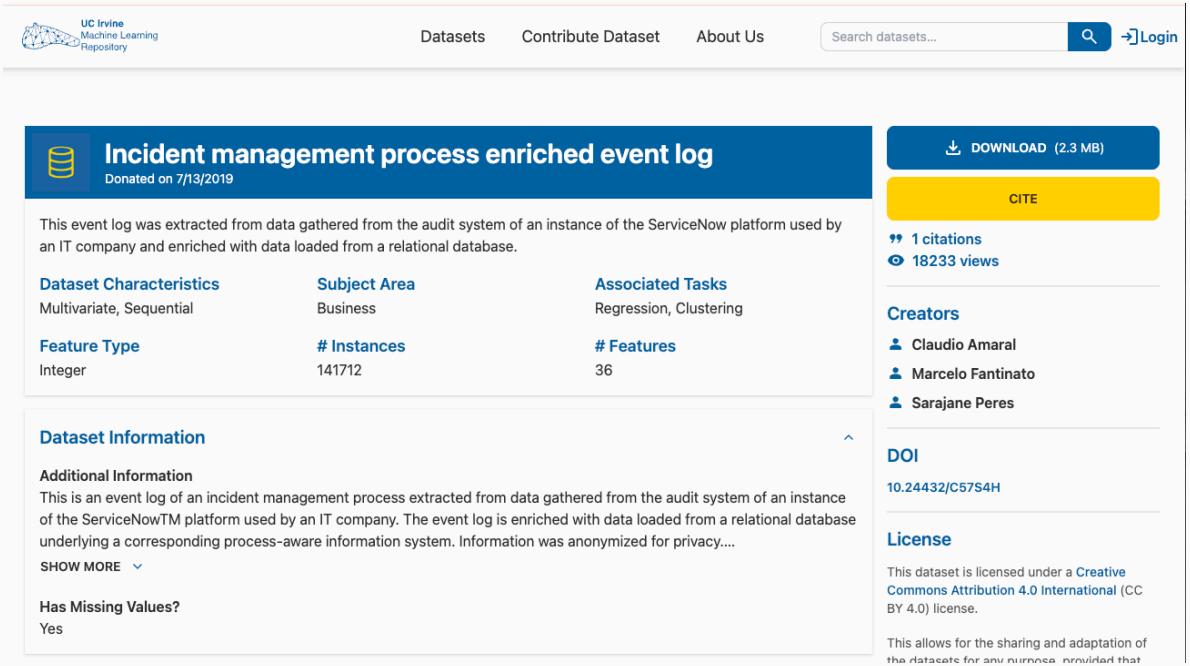
Research Questions

1. **RQ1: Baseline Prediction**
   How accurately can we predict whether an incident will be resolved slowly or quickly?
2. **RQ2: Feature Selection**
   Can we reduce the number of features while maintaining predictive performance?
3. **RQ3: Process Feature Contribution**
   How much do process-related features contribute to predictive performance?
4. **RQ4: Model Optimization**
   Does hyperparameter tuning improve model generalization?

Dataset and Preprocessing

Original Dataset

- **Source:** UCI Machine Learning Repository
- **Link:** *https://archive.ics.uci.edu/dataset/498/incident+management+process+enriched+event+log*
- **Description:**
  - Real-world IT service management dataset
  - Includes attributes such as priority, impact, timestamps, etc



Original Feature Types

- Ticket Attributes:
  - impact, urgency, priority, knowledge, notify
  - SLA status (made_sla), priority confirmation
- Categorical / Identifiers:
  - contact_type, category, subcategory, location
  - assignment_group, assigned_to, caller_id
- Time Information:
  - opened_at, sys_created_at, sys_updated_at
  - resolved_at, closed_at (used to define target)
- Process / Lifecycle Features:
  - reassignment_count, reopen_count, sys_mod_count
  - incident state, assignment changes, updates over time

Data Construction Overview

- Goal:
  - Transform event-level logs into **incident-level dataset**
  - Each incident = one observation for modeling
- Challenges:
  - Multiple rows per incident (event logs)
  - Missing values and noisy features
  - High-cardinality identifiers
- Key steps:
  - Data cleaning and feature removal
  - Event-level $\rightarrow$ incident-level aggregation
  - Feature and target construction

Data Cleaning and Feature Removal

- **Missing values:**
  - Standardized “?” → NaN
- **Removed features:**
  - High missing: *caused_by, vendor, cmdb_ci, rfc*
  - High-cardinality IDs: *caller_id, assigned_to, opened_by*
  - Low information: *active, final_state, closed_code*
  - Hard to encode: *category, subcategory, location*
- **Leakage removal:**
  - *made_sla* depends on resolution outcome → removed

Event-Level to Incident-Level Aggregation

- Grouping:
  - Group by incident ID (*number*)
  - Create one row per incident
- Aggregation strategy:
  - Initial attributes → first value
  - Final attributes → last value
  - Count features → maximum value
- Examples:
  - First: *impact, priority*
  - Last: *resolved_at, closed_at*
  - Max: *reassignment_count, sys_mod_count*

Feature Construction

- Process complexity features:
  - *num_events* = number of records per incident
  - *num_unique_states* = number of unique *incident_state*
  - *num_assignment_groups* = number of unique *assignment_group*
- Process intensity features:
  - *mod_per_event* = *sys_mod_count / num_events*
  - *reassign_per_event* = *reassignment_count / num_events*
- Process indicator features:
  - *was_reassigned* = 1 if reassigned, else 0
  - *was_reopened* = 1 if reopened, else 0
  - *multi_group* = 1 if multiple groups involved, else 0
- Time features (from **opened_at**):
  - *opened_hour*, *opened_dayofweek*, *opened_month*
  - *opened_on_weekend* = 1 if weekend, else 0

Target Variable Construction

- Resolution time:
  - *resolution_time = resolved_at - opened_at*
- Avoiding data leakage:
  - Train/test split performed first
  - All thresholds computed using **training data only**
- Priority-based thresholds:
  - Median resolution time computed **per priority level**
  - Different thresholds for different severity levels
- Final target:
  - *slow_resolution* = 1 if

    resolution_time > median (same priority, train set)

  - Otherwise = 0
- Key idea:
  - "Slow" is defined **relative to incident priority**, not globally

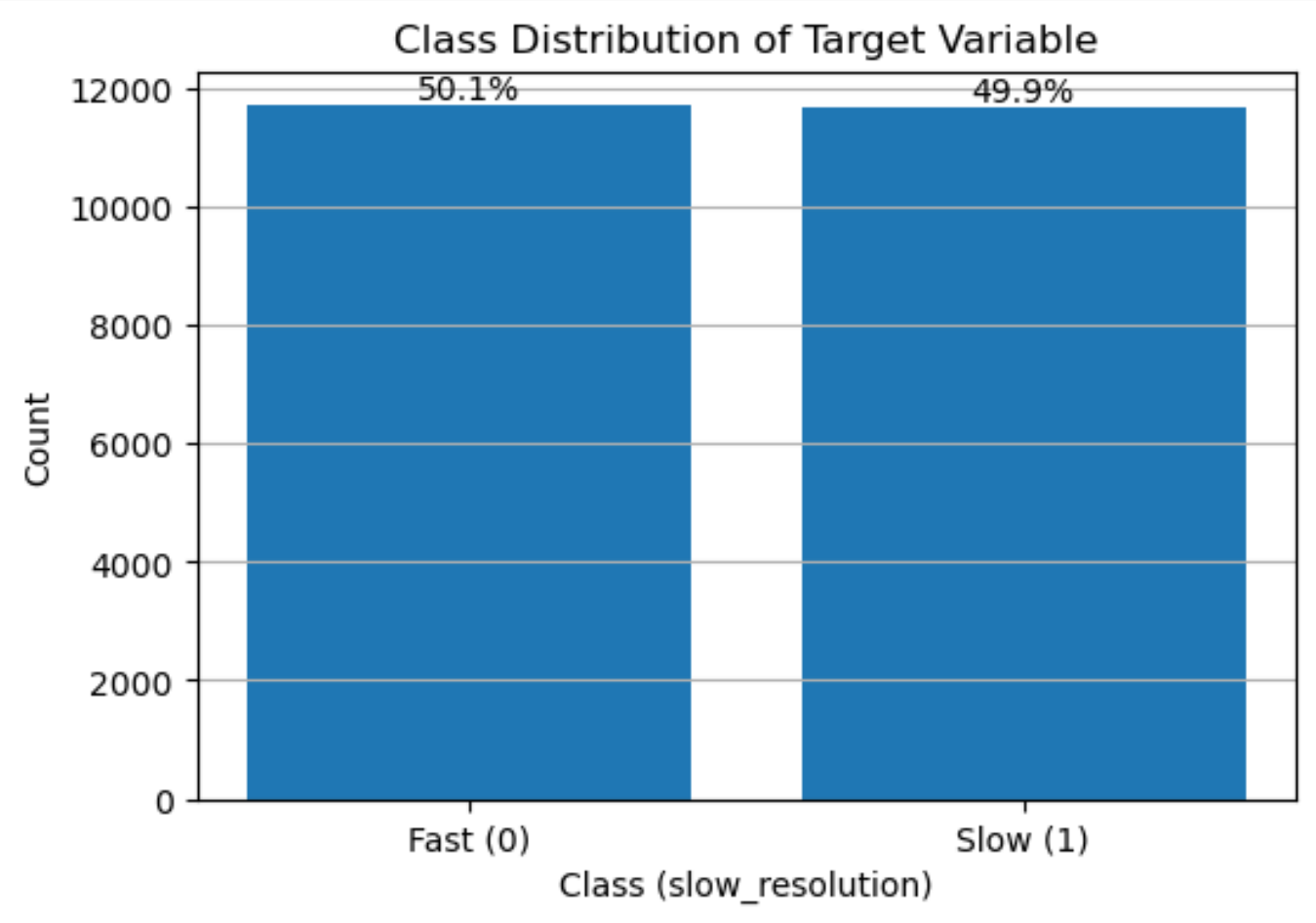
Preprocessing Pipeline

- **Dataset size:**
  - Original (event-level): ∼140,000+ records
  - Final (incident-level): 23,362 incidents
  - Train/Test split: 18,689 / 4,673
- **Encoding:**
  - Binary features → 0/1
  - Ordinal features (*impact, urgency, priority*) → ordered numeric values
- **Final feature space:**
  - Total processed features: 25

Exploratory Data Analysis

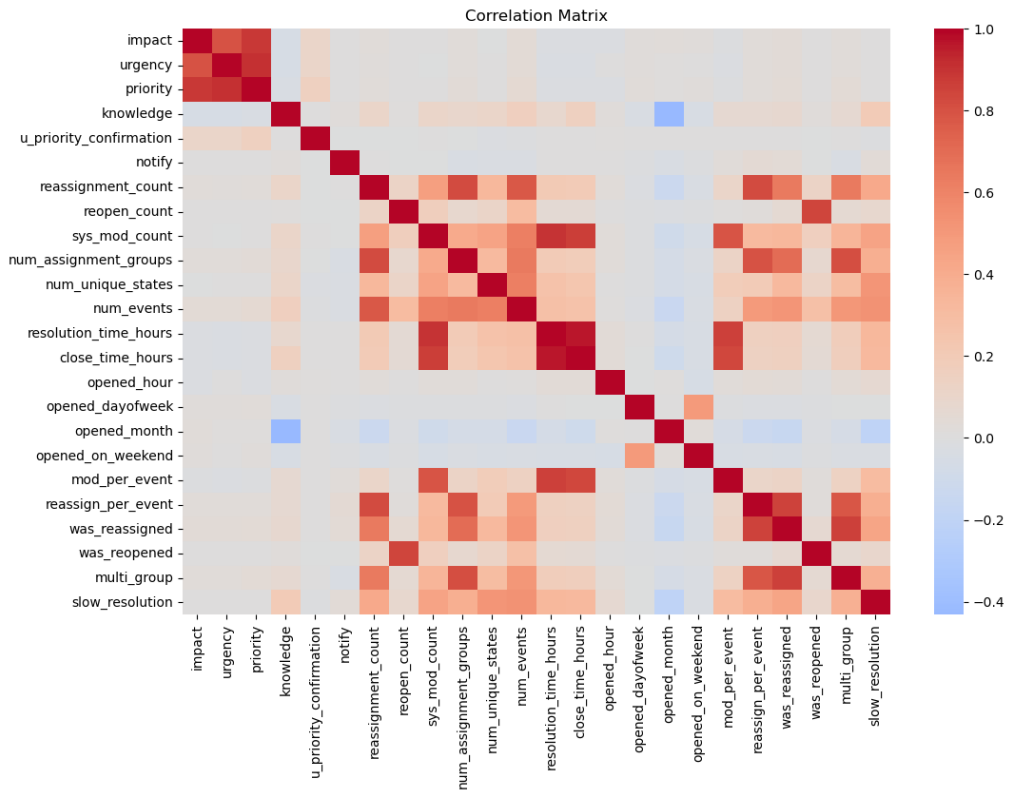
Class Distribution

- Target classes (Fast / Slow) are **balanced**
- Standard metrics (Accuracy, F1) are appropriate



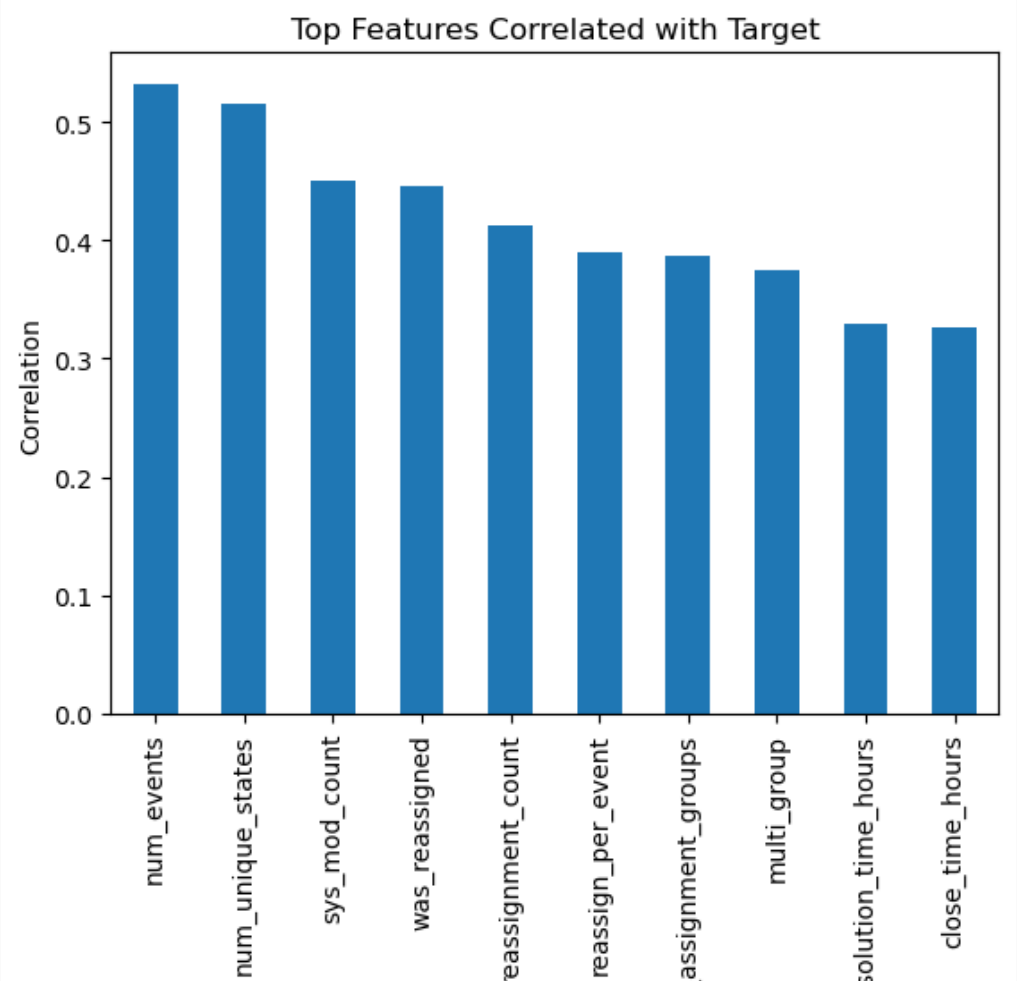
Correlation Analysis (Features)

- Focus on **top highly correlated pairs**
- Examples:
  - *urgency* vs *priority*
  - *reassign_per_event* vs *was_reassigned*
- High correlation → motivates feature selection



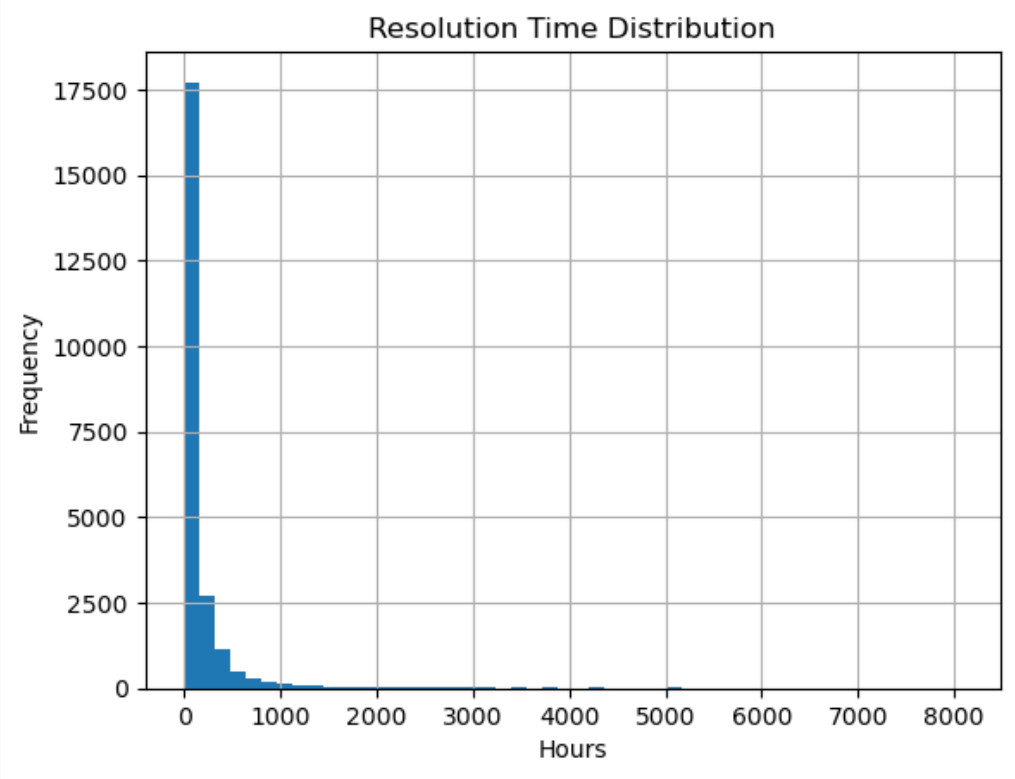
Correlation Analysis (Features with Target)

- Identify **most predictive features**
- Key observations:
  - Process features show **strongest correlation**
  - Priority-related features have **weaker correlation**



Resolution Time Distribution

- Resolution time is **highly right-skewed**
- Most incidents are resolved quickly
- A small number of cases take **very long time** (long tail)
- Implications:
  - Mean is not representative → use **median-based thresholds**



RQ1: Baseline Prediction

Baseline Model Setup

- Three baseline classifiers using all 25 features
- All numeric features are standardized using *`StandardScaler`*
- **Models:**
  - **Logistic Regression**: simple, linear baseline
  - **Decision Tree**: captures non-linear relationships
  - **Random Forest**: ensemble model, handles complex patterns and feature interactions
- **Key characteristics:**
  - LR: assumes linear decision boundary
  - Tree-based models: capture non-linearity and feature interactions

Baseline Results

Model	Features	Accuracy	Macro F1	ROC-AUC
Logistic Regression	25	**0.856**	**0.856**	0.931
Decision Tree	25	0.854	0.854	0.928
Random Forest	25	0.853	0.853	**0.934**

- All models perform similarly across metrics
- Logistic Regression remains simple and competitive
- Random Forest achieves the highest ROC-AUC

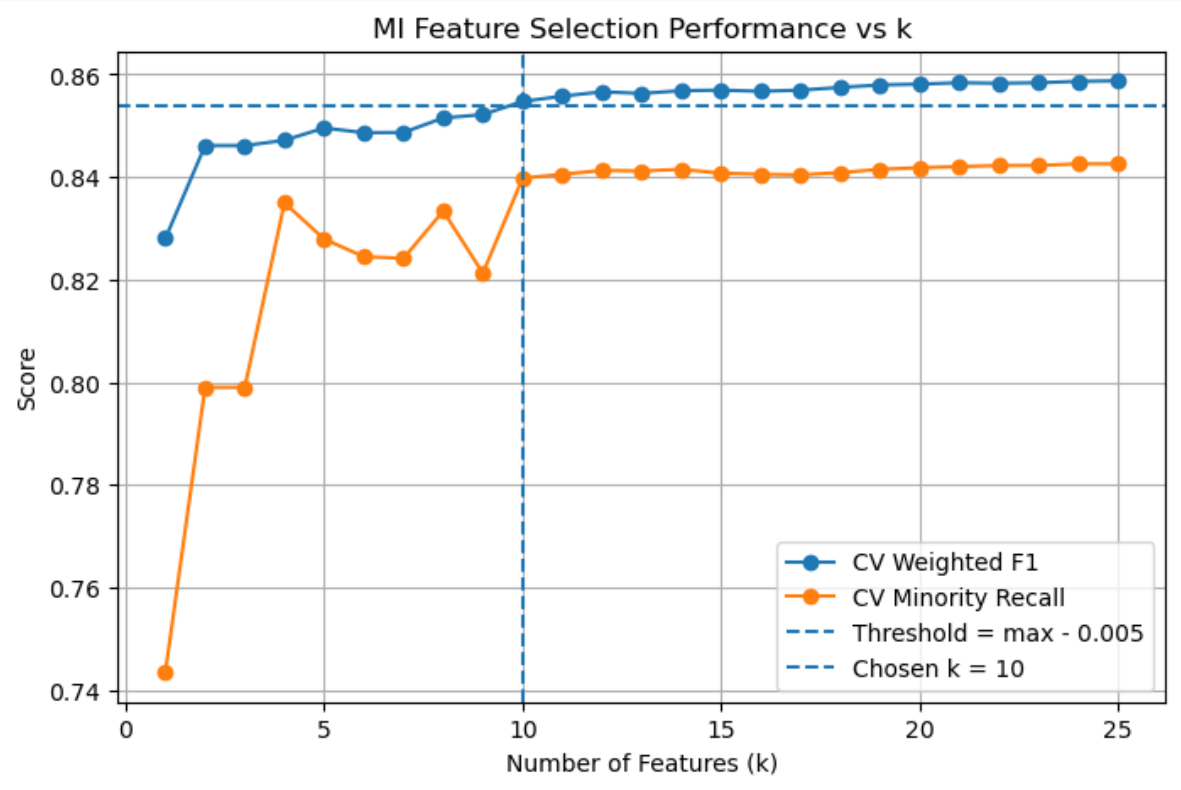
RQ2: Feature Selection

Feature Selection Motivation

- The dataset contains redundant and correlated features
- Goal: reduce dimensionality while preserving performance
- **Methods:**
  - **Mutual Information**
    - Measures dependency between each feature and target
    - Captures **non-linear relationships**
  - **SelectFromModel (Random Forest)**
    - Uses **feature importance** from a trained model
    - Handles **correlated features** better
    - Captures **feature interactions**

Mutual Information

- Number of selected features: **10** (chosen via cross-validation)
- **Correlation reduction:**
  - High-correlation pairs reduced from 10 to **2** after MI
- One-hot encoded *`contact_type`* features are kept as a group



Mutual Information Results

Model	Features	Accuracy	Macro F1	ROC-AUC
Logistic Regression	10	**0.853**	**0.852**	0.927
Decision Tree	10	0.850	0.850	0.928
Random Forest	10	0.852	0.852	**0.930**

- Performance remains comparable with fewer features
- Logistic Regression remains high performance
- Random Forest achieves highest ROC-AUC

SelectFromModel

- Uses model-based feature importance from Random Forest
- Captures **nonlinear relationships and feature interactions**
- **Feature selection:**
  - Importance threshold = **median importance**
  - Features above this threshold are kept
  - Final selected features: **13**
- **Correlation reduction:**
  - High-correlation pairs reduced from 10 to **2**

SelectFromModel Results

Model	Features	Accuracy	Macro F1	ROC-AUC
Logistic Regression	13	**0.855**	**0.855**	0.930
Decision Tree	13	0.851	0.851	0.928
Random Forest	13	0.854	0.854	**0.933**

- Performance remains comparable with 13 features
- Logistic Regression remains strong and stable
- Random Forest achieves highest ROC-AUC

Logistic Regression Results Across Feature Sets

Feature Set	Features	Accuracy	Macro F1	ROC-AUC
Baseline	25	**0.856**	**0.856**	**0.931**
SelectFromModel	13	0.855	0.855	0.930
MI	10	0.853	0.853	0.927

- Feature reduction with minimal performance loss
- SelectFromModel is closest to baseline

Decision Tree Results Across Feature Sets

Feature Set	Features	Accuracy	Macro F1	ROC-AUC
Baseline	25	**0.854**	**0.854**	**0.928**
SelectFromModel	13	0.851	0.851	0.928
MI	10	0.850	0.850	0.928

- Slight performance drop after feature selection
- ROC-AUC remains stable across feature sets

Random Forest Results Across Feature Sets

Feature Set	Features	Accuracy	Macro F1	ROC-AUC
Baseline	25	0.853	0.853	**0.934**
SelectFromModel	13	**0.854**	**0.854**	0.933
MI	10	0.852	0.852	0.930

- SelectFromModel slightly improves accuracy and Macro F1
- Baseline achieves highest ROC-AUC

RQ3: Process Feature Contribution

What Are Process Features?

- Process features describe how an incident evolves over time.
- They are not defined by whether they are original or engineered.
- Instead, they are defined by what they represent.
- Examples:
  - *reassignment_count*
  - *reopen_count*
  - *sys_mod_count*
  - *num_events*
  - *num_unique_states*
  - *mod_per_event*

Why Remove Process Features?

- Process features may contain strong predictive signal.
- However, some process features may only be available after the ticket has evolved.
- Removing them helps test whether static initial ticket information is enough.
- This experiment isolates the contribution of lifecycle-based information.

No Process Feature Results

Model	Features	Accuracy	Macro F1	ROC-AUC
Logistic Regression	14	0.630	**0.627**	0.656
Decision Tree	14	0.624	0.621	0.656
Random Forest	14	**0.636**	0.627	**0.668**

- Removing process features causes a **significant performance drop** (≈ 0.85 → 0.63)
- Process features contain the **major predictive signal**
- Initial/static features alone are **not sufficient**

RQ4: Model Optimization

Optimization Setup

- Hyperparameter tuning is performed only on the full-feature baseline models.
- GridSearchCV is used with cross-validation on the training set.
- Main parameters:
  - Logistic Regression: regularization strength and penalty mix
  - Decision Tree: max depth, minimum samples per leaf, split criteria
  - Random Forest: number of trees, max depth, minimum samples per leaf

Optimized Model Results

Model	Features	Accuracy	Macro F1	ROC-AUC
Logistic Regression	25	0.856	0.856	0.931
Decision Tree	25	0.851	0.851	0.925
Random Forest	25	**0.860**	**0.860**	**0.936**

- Hyperparameter tuning results in **minor improvements**
- Random Forest achieves the best overall performance
- Gains are relatively small compared to baseline

Final Comparison

Final Model Comparison

Feature Set	Model	Features	Accuracy	Macro F1	ROC-AUC
Baseline	LR	25	0.856	0.856	0.931
Baseline	DT	25	0.854	0.854	0.928
Baseline	RF	25	0.853	0.853	0.934
SelectFromModel	LR	13	0.855	0.855	0.930
SelectFromModel	DT	13	0.851	0.851	0.928
SelectFromModel	RF	13	0.854	0.854	0.933
MI	LR	10	0.853	0.852	0.927
MI	DT	10	0.850	0.850	0.928
MI	RF	10	0.852	0.852	0.930
No Process	LR	14	0.630	0.627	0.656
No Process	DT	14	0.624	0.621	0.656
No Process	RF	14	0.636	0.627	0.668
Optimized Baseline	LR	25	0.856	0.856	0.931
Optimized Baseline	DT	25	0.851	0.851	0.925
Optimized Baseline	RF	25	**0.860**	**0.860**	**0.936**

- Best overall model: **Optimized Random Forest**
- Feature selection reduces features with minimal performance loss

Final Conclusion

- **RQ1: Can we predict resolution time effectively?**
  Yes — all models achieve strong performance (Accuracy / Macro F1 ≈ 0.85)
- **RQ2: Do fewer features maintain performance?**
  Yes — reducing from 25 $\rightarrow$ 10–13 features keeps performance nearly unchanged
- **RQ3: Do process features matter?**
  Yes — removing them causes a large drop ($\approx 0.85 \rightarrow 0.63$), showing they dominate prediction
- **RQ4: Does model optimization help?**
  Slightly — tuning improves Random Forest marginally, but gains are limited
- **Key insight:**
  - Prediction is **easy with process features**, but **hard without them**
  - Model performance is driven more by **feature informativeness** than model choice

Limitations

- Process features may not be available at early prediction time
- Results may not generalize to different IT systems
- Binary classification simplifies a continuous outcome

Future Work

- Develop early-stage prediction models without process features
- Explore time-series or sequential models (e.g., LSTM)
- Incorporate textual data (incident descriptions)
- Predict exact resolution time instead of binary classification

Questions?